# UAV Software Engineering: ROS 2 Architecture

The DDS Protocol and Publish/Subscribe Pattern

## 1 The Middleware: Data Distribution Service (DDS)

Historically, robotic frameworks relied on centralized brokers (e.g., ROS 1's `roscore`). In mission-critical UAV systems, a centralized broker is a Single Point of Failure (SPOF). ROS 2 mitigates this by abstracting its communication layer over the **Data Distribution Service (DDS)**, an Object Management Group (OMG) standard designed for real-time, deterministic, and distributed systems.

DDS operates purely Peer-to-Peer (P2P). Nodes discover each other dynamically using the Real-Time Publish Subscribe (RTPS) protocol over UDP multicast. Once discovered, Publishers and Subscribers establish direct data channels, eliminating the need for a central master.

## 2 The Publish-Subscribe Architecture

DDS expands on the Producer-Consumer pattern implemented in multi-threading. Instead of sharing a memory queue, nodes communicate asynchronously via **Topics**. A Publisher writes data to a Topic, and multiple Subscribers can read from it without knowing the Publisher's identity or location. This strict decoupling allows developers to hot-swap or scale subsystems (e.g., swapping a physical camera node with a SITL simulation node) without altering the core flight controller.

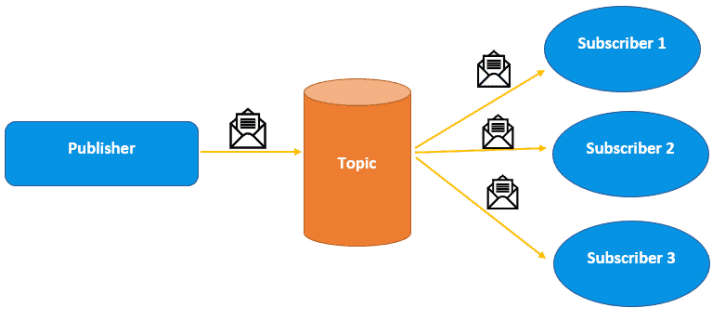



Figure 1: Publish-Subscribe Architecture and access through Topic.

## 3 OMG Specification and DDS Vendors

DDS is not a software framework but a formal specification created by the **Object Management Group (OMG)** for real-time, mission-critical systems. Because it is an open standard, ROS 2 relies on third-party implementations (DDS Vendors) such as eProsima Fast DDS, Eclipse Cyclone DDS, or RTI Connext. This allows robotics engineers to swap the underlying network engine based on certification requirements (e.g., DO-178C for aerospace) without rewriting application logic.

## 4 Applicative Architecture: DCPS and Domains

Unlike message-centric protocols, DDS employs a **Data-Centric Publish-Subscribe (DCPS)** model. It creates the illusion of a *Global Data Space (GDS)* which is a distributed, shared memory database. Publishers write updates to this space, and subscribers read from it.

To isolate traffic on the same physical network (e.g., running a simulation on the same Wi-Fi as a real drone), DDS uses **Domain IDs**. Only nodes (DomainParticipants) sharing the same Domain ID can discover each other and share data. Data structures are strictly defined using the **Interface Definition Language (IDL)**, ensuring memory alignment across different architectures (e.g., x86 ground stations communicating with ARM-based flight controllers).

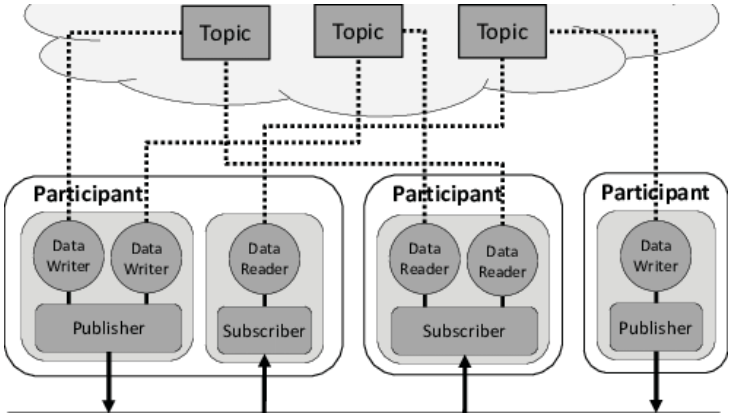



Figure 2: Publish-Subscribe Architecture and access through Topic.

# 5 The Wire Protocol: RTPS and Discovery

Under the hood, DDS utilizes the **Real-Time Publish-Subscribe (RTPS)** wire protocol, typically over UDP. The decentralized network avoids Single Points of Failure through a two-phase discovery mechanism:

1. **Participant Discovery (SPDP):** Nodes broadcast UDP multicast packets to announce their presence on a specific Domain ID.
2. **Endpoint Discovery (SEDP):** Once participants discover each other, they use UDP unicast to exchange their Topics, Data Types, and QoS profiles. If the QoS profiles are compatible (e.g., a Reliable publisher matched with a Reliable subscriber), the data connection is established.

While SPDP relies on UDP multicast, complex UAV networks (e.g., BVLOS flights over LTE or ground stations behind VPNs) often block multicast traffic. To solve this, DDS implementations offer **Unicast Discovery** via Initial Peers lists or centralized **Discovery Servers**, allowing nodes to bypass multicast and connect directly via known IP addresses.

# 6 ROS 2 Abstraction: The RMW Layer

ROS 2 abstracts the complexity of DDS via the **ROS Middleware (RMW)** interface. Developers write C++ code using the ROS Client Library (`rclcpp`). The RMW layer translates these API calls into the specific vendor's DDS functions. This abstraction allows instantaneous swapping of the DDS implementation by simply changing the `RMW_IMPLEMENTATION` environment variable, ensuring extreme modularity in UAV software stacks.

# 7 Quality of Service (QoS) Policies for UAVs

The defining feature of DDS is its Quality of Service (QoS) profiles, which allow developers to tune the network behavior to the physical constraints of drone data links (e.g., lossy telemetry radios). Key policies include:

- **Reliability:**
  - *Best Effort:* The publisher sends messages once. Ideal for high-frequency sensor data (IMU, GPS) where stale data is useless and bandwidth is scarce.

  - *Reliable:* The publisher retransmits unacknowledged packets. Mandatory for critical state commands (e.g., Arming, Mission Waypoints).

- **Durability:**
  - *Volatile:* Late-joining subscribers only receive data published after they connect.
  - *Transient Local:* The publisher saves a history of messages and sends them to late-joining subscribers. Useful for static environment data (e.g., map configurations).

- **History:** Defines how many messages to keep in the queue before overwriting them (e.g., `KEEP_LAST` with depth $N$, or `KEEP_ALL`).

- **Liveliness:** Determines how the system monitors if a node is still active.
  - *Automatic:* The DDS middleware automatically asserts liveliness as long as the node exists.
  - *Manual By Topic:* The application must explicitly publish data or assert liveliness within a defined lease duration. If a flight controller stops receiving liveliness assertions from a sensor node, it can immediately trigger a hardware failsafe.